

PROMPT ENGINEERING

A Practical Course for AI Engineers

Writing effective prompts for Claude, ChatGPT (OpenAI), and other large language models

Course Contents

- Module 1 — Foundations of Prompt Engineering
- Module 2 — Anatomy of a Great Prompt
- Module 3 — Core Prompting Techniques
- Module 4 — Prompting Claude (Anthropic)
- Module 5 — Prompting ChatGPT / GPT models (OpenAI)
- Module 6 — Claude vs GPT: Key Differences
- Module 7 — Prompting for AI Engineers (API-level techniques)
- Module 8 — Common Mistakes and How to Fix Them
- Module 9 — Practice Exercises
- Module 10 — Quick Reference Cheat Sheet

Module 1: Foundations of Prompt Engineering

Prompt engineering is the skill of writing instructions (prompts) that guide a large language model (LLM) such as Claude or ChatGPT to produce the output you want. The model does not read your mind — it responds to the exact words, structure, and context you give it. Better input almost always means better output.

Why Prompt Engineering Matters for an AI Engineer

- It directly affects accuracy, consistency, and cost (fewer retries, fewer tokens wasted).
- It lets you control output format, so it can be parsed programmatically (JSON, XML, tables).
- It reduces hallucination by giving the model the right context and constraints.
- It is often faster and cheaper than fine-tuning a model for a new task.

How LLMs Actually Process a Prompt

An LLM predicts the next token based on everything before it. This means the prompt is not just a question — it is the entire context the model uses to decide what to generate next. Order, wording, examples, and even formatting (headings, bullet points, XML tags) all change the output.

Key idea: Think of a prompt as a specification, not a casual question. The more precisely you specify the task, the more reliable the result.

Module 2: Anatomy of a Great Prompt

Most strong prompts are built from the same building blocks. You do not need all of them every time, but knowing them lets you diagnose why a prompt is not working.

Component	Purpose	Example
Role / Persona	Sets tone, expertise level, and perspective	"You are a senior backend engineer reviewing code."
Task	The specific action you want performed	"Summarize this document in 3 bullet points."
Context	Background information the model needs	"This is a support ticket from a SaaS product."
Constraints	Rules, format, length, tone limits	"Respond in under 100 words, in JSON format."

Component	Purpose	Example
Examples	Sample input/output pairs (few-shot)	"Input: ... Output: ..."
Output format	Exact structure expected	"Return only valid JSON with keys: name, score."

Simple Before/After Example

Weak prompt: "Write about dogs."

Strong prompt: "You are a veterinary content writer. Write a 150-word blog intro about why Labrador Retrievers make good family pets. Use a warm, friendly tone. Do not use bullet points."

Why it works: The strong version defines the role, topic, length, tone, and format — leaving almost no room for the model to guess wrong.

Module 3: Core Prompting Techniques

1. Zero-shot Prompting

You ask the model to perform a task with no examples, relying on its pretrained knowledge.

```
Classify the sentiment of this review as Positive, Negative, or Neutral:  
"The delivery was late but the product quality is excellent."
```

2. Few-shot Prompting

You give the model 2-5 examples of input/output pairs before the real task. This is one of the most reliable ways to control format and style.

```
Review: "Fast shipping, loved it!" -> Positive  
Review: "Arrived broken, terrible experience." -> Negative  
Review: "It was okay, nothing special." -> Neutral  
Review: "The delivery was late but the product quality is excellent." -> ?
```

3. Chain-of-Thought (CoT) Prompting

You ask the model to reason step by step before giving a final answer. This improves accuracy on math, logic, and multi-step tasks.

```
Question: A store had 120 items. It sold 35% on Monday and 20 items on Tuesday.  
How many items are left?  
Think step by step, then give the final answer on the last line as "Answer: X".
```

4. Role / Persona Prompting

Assigning a role focuses the model's tone, vocabulary, and depth of reasoning.

```
You are a senior data scientist mentoring a junior engineer.  
Explain overfitting in simple terms, using one real-world analogy.
```

5. Instruction + Format Prompting

Explicitly stating the desired output structure (JSON, table, markdown) dramatically reduces parsing errors in production systems.

```
Extract the name, email, and phone number from the text below.  
Return ONLY valid JSON with keys: name, email, phone.  
If a field is missing, use null.  
  
Text: "Hi, I am Ali Khan, reach me at ali@email.com."
```

6. Prompt Chaining

Instead of one giant prompt, break a complex task into smaller prompts, where the output of one step becomes the input of the next. This is common in AI-engineering pipelines (e.g., RAG systems, agents).

- Step 1: Extract key facts from a document.
- Step 2: Summarize the extracted facts.
- Step 3: Generate a final report from the summary.

Module 4: Prompting Claude (Anthropic)

Claude was trained to work especially well with clear structure and XML-style tags. Anthropic's own prompt engineering guidance emphasizes being explicit, giving Claude room to think, and using tags to separate different parts of a prompt.

Best Practices for Claude

- **Be explicit and direct.** Claude follows literal instructions closely — state exactly what you want, including what NOT to do.
- **Use XML tags to structure prompts.** Tags like <document>, <instructions>, and <example> help Claude clearly separate context from instructions.

- **Give Claude a role via the system prompt.** Use the system parameter (in the API) to set persona and global behavior rather than repeating it every message.
- **Ask for step-by-step reasoning** when the task is complex, using a `<thinking>` tag before the final answer.
- **Use multishot examples** wrapped in tags for consistent formatting.
- **Prefill the response** (API only) to force a starting format, e.g. starting the assistant turn with "{" to force JSON.

Example: Structured Claude Prompt with XML Tags

```
<document>
{{PASTE_LONG_DOCUMENT_HERE}}
</document>

<instructions>
Summarize the document above in exactly 3 bullet points.
Focus only on financial figures. Do not include opinions.
</instructions>
```

Example: Using a System Prompt (Claude API)

```
system: "You are a precise financial analyst. Always cite numbers exactly as
given. Never guess a number that is not in the source text."

user: "<document>...</document>\n\nWhat was Q2 revenue?"
```

Tip: Claude responds very well to being told the reasoning process explicitly, e.g. "First list the relevant facts, then answer in one sentence."

Module 5: Prompting ChatGPT / GPT Models (OpenAI)

GPT models (via ChatGPT or the OpenAI API) respond well to clear role separation using the system / user / assistant message roles, and to direct, well-structured natural-language instructions. They tend to be more flexible with plain markdown formatting than strict XML tags.

Best Practices for GPT Models

- **Use the system message** to set persona, tone, and rules that should apply to the whole conversation.
- **Be specific about format** — GPT models respond well to markdown headers, numbered steps, and JSON schemas.
- **Use delimiters** like triple quotes, dashes, or markdown code fences to separate instructions from content.

- **Ask for a plan before the answer** for complex reasoning tasks (similar to chain-of-thought).
- **Use function calling / structured outputs** (API feature) when you need guaranteed JSON matching a schema.
- **Iterate with follow-up prompts** — GPT models handle multi-turn refinement well ("make it shorter", "now add a table").

Example: Structured GPT Prompt

```
SYSTEM: You are a customer support assistant for an e-commerce app. Always  
reply in a polite, concise tone.
```

```
USER:
```

```
"""
```

```
Customer message: "My order #4521 has not arrived after 10 days."
```

```
"""
```

```
Write a reply that:
```

- 1) Apologizes briefly
- 2) Asks for the order confirmation email
- 3) Gives an estimated resolution time of 48 hours

Example: Requesting JSON Output (OpenAI API)

```
Extract the following fields from the text and return valid JSON only:  
fields: order_id, issue_type, urgency (low/medium/high)
```

```
Text: "Order #4521 still hasn't shipped after 10 days, I need this urgently."
```

Tip: When using the OpenAI API, prefer the built-in "structured outputs" / JSON schema feature over asking nicely for JSON in plain text — it guarantees valid, schema-matching output.

Module 6: Claude vs GPT: Key Differences in Prompting Style

Aspect	Claude (Anthropic)	ChatGPT / GPT (OpenAI)
Structuring prompts	Prefers XML-style tags (<context>, <task>)	Prefers markdown, delimiters, numbered lists
System prompt	Strong adherence; good for persona + hard rules	Strong adherence; good for persona + tone

Aspect	Claude (Anthropic)	ChatGPT / GPT (OpenAI)
Long documents	Very strong at long-context, structured documents	Strong, especially with clear section breaks
Step-by-step reasoning	Responds well to explicit "think step by step" or a thinking tag	Responds well to "show your reasoning, then answer"
Forcing exact format	Prefill technique (start assistant reply) in API	Structured Outputs / JSON schema in API
Tone following instructions	Very literal — say exactly what you want, including exceptions	Flexible but benefits from explicit constraints too

Bottom line: The core skill — being specific, giving context, defining format — transfers across both models. The main difference is packaging: Claude leans toward tagged structure, GPT leans toward markdown and schema-based function calling.

Module 7: Prompting for AI Engineers: API-Level Techniques

As an AI engineer, you will mostly interact with these models through APIs, not chat UIs. A few extra techniques matter at this level.

1. System vs User vs Assistant Roles

Always put stable rules and persona in the system message/prompt, and put the changing task/content in the user message. This keeps prompts modular and reusable across requests.

2. Temperature and Sampling Parameters

- **Low temperature (0–0.3):** use for factual tasks, extraction, classification, code generation.
- **Higher temperature (0.7–1.0):** use for creative writing, brainstorming, varied outputs.
- **top_p:** an alternative way to control randomness; usually tune one, not both, at a time.

3. Function Calling / Tool Use

Both Claude and OpenAI support giving the model a set of "tools" (functions) it can call with structured arguments. This is essential for building agents that query databases, call APIs, or run code. Define tool schemas clearly, with descriptive names and parameter descriptions — the model relies on these descriptions to decide when and how to call a tool.

4. Retrieval-Augmented Generation (RAG) Prompts

When injecting retrieved documents into a prompt, clearly separate retrieved context from the question, and instruct the model to answer only from the provided context to reduce hallucination.

```
<context>
{{RETRIEVED_CHUNKS}}
</context>
```

Using ONLY the information in the context above, answer the question below.
If the answer is not in the context, say "I don't have enough information."

Question: {{USER_QUESTION}}

5. Evaluating and Iterating on Prompts

- Build a small test set of representative inputs and expected outputs.
- Change one variable at a time (wording, examples, format) and compare results.
- Track token usage and latency alongside quality — a great prompt that is too expensive is not production-ready.
- Version your prompts like code (store them in files, track changes in git).

Module 8: Common Mistakes and How to Fix Them

Mistake	Problem	Fix
Vague instructions	Model guesses what you want	State task, format, and length explicitly
No examples for format-sensitive tasks	Inconsistent output structure	Add 2-3 few-shot examples
Mixing instructions and data with no separation	Model confuses content for commands	Use tags/delimiters to separate them
Asking for too much in one prompt	Lower quality, missed steps	Break into a prompt chain (multi-step)
Not specifying what to do when info is missing	Model hallucinates an answer	Add "if unknown, say so" instruction
Ignoring system prompt for stable rules	Rules get diluted or forgotten in long chats	Move persona/rules to the system prompt

Module 9: Practice Exercises

Try rewriting each weak prompt below into a strong one, using the techniques from this course. Practice on both Claude and ChatGPT to compare results.

- 1 Weak: "Summarize this." — Rewrite it with role, format, and length constraints.
- 2 Weak: "Write code for a login page." — Rewrite it specifying language, framework, and edge cases to handle.
- 3 Weak: "Is this email spam?" — Rewrite it as a few-shot classification prompt with 3 examples.
- 4 Weak: "Translate this." — Rewrite it specifying source/target language, tone, and formatting rules.
- 5 Weak: "Fix my resume." — Rewrite it with a role (recruiter), specific criteria, and desired output structure.

Practice tip: For each exercise, run both your weak and strong version on the same model and compare the outputs side by side. This builds intuition faster than reading theory alone.

Module 10: Quick Reference Cheat Sheet

The 5-Step Prompt Checklist

- 1 **Role** — Who should the model act as?
- 2 **Task** — What exact action should it perform?
- 3 **Context** — What background does it need?
- 4 **Format** — What should the output look like?
- 5 **Constraints** — What are the limits (length, tone, things to avoid)?

Claude Quick Tips

- Use XML tags: <context>, <task>, <example>, <thinking>.
- Put persona and hard rules in the system prompt.
- Be explicit — Claude follows instructions very literally.

ChatGPT / GPT Quick Tips

- Use markdown structure and triple-quote delimiters.
- Use system messages for persona and tone.
- Use Structured Outputs / JSON schema for guaranteed formats in the API.

End of Course. Keep a personal prompt library of your best-performing prompts — it becomes one of the most valuable assets you build as an AI engineer.